


# Software Development Plan

Next-Generation Ground-based Solar Observing Network

ngGONG Software Team

2026-10-30

# Table of contents


1 Purpose and Scope 3

2 Software Organization and Responsibilities 4

3 Software Development Life Cycle 5

4 Requirements Management and Traceability 6

5 Interfaces, Architecture, and Design 7

6 Development Environment and Tooling 8

7 Source and Configuration Management 9

8 Coding Standards and Review Practices 10

9 Build, Integration, and Continuous Integration 11

10 Verification and Validation 12

11 Release and Deployment Management 13
- 11.1 Controls . . . 13
- 11.2 DMS . . . 14
- 11.3 HLS - OCS . . . 14
- 11.4 HLS - SCS . . . 14

12 Defect and Issue Management 16

13 Cybersecurity Coordination 17

14 Transition to Operations and Software Maintenance 18

# 1 Purpose and Scope

The Software Development Plan (SDP) describes the project-level approach for planning, developing, controlling, integrating, verifying, releasing, and maintaining software for the Next-Generation Ground-based Solar Observing Network (ngGONG).

For the Conceptual Design Review (CoDR), this document establishes the intended software development framework and identifies areas where detailed standards, tools, and procedures will be defined as the project matures. Over time, this plan is expected to evolve into a working standard for ngGONG software teams.

This plan applies to ngGONG software delivered under Work Breakdown Structure (WBS) 9.0, High Level Software and Controls, and WBS 10.0, Data Management System. It describes software practices that are expected to be common across High Level Software, Controls, and the Data Management System, including requirements traceability, development workflow, configuration management, review, build, integration, verification, release, and maintenance.

Where High Level Software and Controls, the Data Management System, or other software areas require different tools or detailed procedures, those details may be defined in subordinate standards, repository-specific documentation, interface control documents, test plans, deployment procedures, or operations procedures.

This plan does not define project-level processes for document control, risk management, project reporting, budgeting, scheduling, or formal change control. Those processes are managed through ngGONG project-level systems engineering, project management, configuration management, and risk management plans. The software team participates in those processes where software work products, risks, changes, progress, or verification evidence are involved.

This plan also does not define the detailed software architecture, subsystem designs, interface specifications, coding standards, cybersecurity implementation, deployment topology, test procedures, or operations runbooks. Those artifacts are maintained separately. This plan identifies where such artifacts are needed and describes how they fit into the overall software development process.

# 2 Software Organization and Responsibilities

ngGONG software is currently developed and planned by a small core team covering WBS 9.0, High Level Software and Controls, and WBS 10.0, Data Management System. During early design, one person may cover multiple software functions, including planning, technical direction, implementation, interface coordination, integration, verification, release planning, and maintenance planning.

As the project matures, software responsibilities will be refined to reflect staffing, contractor scope, subsystem ownership, and operational needs. Contractor-delivered software packages are expected to follow applicable ngGONG software practices and to have an assigned ngGONG technical point of contact or software owner.

Coordination across the WBS 9.0 and WBS 10.0 boundary is the responsibility of the applicable software leads. Project-level authority for staffing, budget, formal change control, risk management, document control, and project reporting is defined outside this plan.

# 3 Software Development Life Cycle

ngGONG software development will use an iterative and incremental life cycle. During early design, the life cycle will emphasize requirements definition, architecture, interface definition, risk-reduction prototypes, simulations, and development infrastructure. Prototypes will be used to evaluate uncertain or high-impact technical areas before the project commits to detailed designs or implementation patterns. Early development infrastructure will include the build, test, integration, and deployment mechanisms needed to support later implementation work. As the project matures, the emphasis will shift toward implementation, integration, verification evidence, release control, deployment readiness, and operations support.

The expected development flow is requirements, design, implementation, review, build, integration, verification, release, deployment, and feedback from test, commissioning, and operations. The flow is not expected to be strictly linear. Design decisions, test results, defects, interface changes, and operational feedback may cause earlier work to be revisited.

ngGONG intends to use a Continuous Integration / Continuous Deployment (CI/CD)-informed development model in which software is built, tested, and integrated frequently. Continuous integration will be used to detect defects, interface changes, and integration problems early. Deployment automation will be used where appropriate, but promotion to site and operational environments will remain controlled by project and operations readiness criteria.

Software integration will occur early and repeatedly, especially where behavior crosses the WBS 9.0 and WBS 10.0 boundary. Simulators, test harnesses, and representative data flows are part of the development life cycle and are expected to mature with the delivered software.

# 4 Requirements Management and Traceability

Software requirements define the behavior and constraints that ngGONG software must satisfy. They may include functional behavior, interfaces, data products, timing, performance, reliability, observability, deployment, maintainability, and operations-support needs.

Software requirements will be derived from the ngGONG Operational Concept Definition, Operational Architecture Document, Science Requirements, Instrument Science Requirements, system and subsystem Design Requirements Documents (DRDs), interface definitions, and operations needs. Additional derived requirements may be identified through design work, prototypes, integration planning, verification planning, and externally developed software.

Software requirements will use stable identifiers so that related software work can reference them. Where applicable, issues, design artifacts, pull requests, code changes, tests, release records, and verification evidence will reference the requirements they satisfy or affect. The project has not yet selected the final tooling or traceability schema, but the software process will preserve links from requirements through implementation and verification using a project-approved traceability schema. The level of traceability should match the risk and importance of the software being changed.

Each software requirement should have an identified verification approach. Verification may be performed by automated test, integration test, analysis, inspection, demonstration, or operational acceptance, depending on the requirement. Where practical, automated tests should be used to support repeatable requirement verification and regression testing.

Traceability is especially important where behavior crosses software subsystem or hardware-control boundaries. This includes interfaces between High Level Software and Controls, where high-level software interacts with low-level programmable logic controller (PLC) and hardware control systems, and interfaces between High Level Software, Controls, and the Data Management System, where data, metadata, status, and operational context are exchanged. These cross-boundary behaviors must be traceable to the requirements and verification evidence that govern them.

Externally developed software may be produced through different contracting models, including staff augmentation, prototype studies, configured products, or delivered software packages. For software that becomes part of ngGONG, the responsible ngGONG software owner must ensure that expectations, interfaces, acceptance criteria, delivered artifacts, and maintenance responsibilities are clear before the software is accepted.

# 5 Interfaces, Architecture, and Design

Interfaces will be defined in forms that can be reviewed, version controlled, and tested. Where practical, ngGONG will use repository-controlled schemas, Protocol Buffer definitions, interface definition files, data models, or API specifications as the authoritative interface definition. The repository that owns an interface definition should produce the generated artifacts needed by both producers and consumers. Prose interface control documents may still be used, but they should explain intent, ownership, compatibility expectations, and operational assumptions rather than duplicate machine-readable definitions.

The interface between High Level Software and Controls is a major design boundary. It includes commands, state models, telemetry, alarms, timing behavior, interlocks, fault handling, and recovery behavior across the boundary between high-level software and low-level Programmable Logic Controller (PLC) and hardware control systems. The interfaces between High Level Software, Controls, and the Data Management System are also controlled design boundaries, especially for data products, metadata, provenance, transfer status, and reduction context.

Design decisions that affect more than one team or subsystem will be captured in a durable form, such as repository documentation, interface definitions, or architecture decision records. Simulators, test harnesses, and representative data flows may be developed as part of the design when hardware, site systems, or Data Management System services are not yet available.

# 6 Development Environment and Tooling

ngGONG will define development environment and tooling expectations as the software architecture and implementation technologies mature. At this stage, the intent is to standardize where practical while allowing High Level Software, Controls, and the Data Management System to use tools that fit their technical needs.

Each software repository should document the tools and environment needed to build, test, and run the software it contains. This may include compilers, runtimes, package managers, test tools, simulators, configuration files, environment variables, credentials, and external services. The goal is that a new developer, contractor, or reviewer can reproduce the development setup without relying on undocumented local knowledge.

Tooling choices should support repeatable build, test, static analysis, packaging, deployment, and integration activities where practical. Scripted, containerized, or otherwise reproducible development environments may be used when they reduce setup drift, improve continuous integration, or make contractor handoff more reliable.

# 7 Source and Configuration Management

ngGONG will use GitHub as the primary source-control platform for delivered software. Software repositories should be maintained under the ngGONG GitHub organization unless a specific exception is needed.

Repository contents are controlled software artifacts. This includes source code, tests, build definitions, interface definitions, configuration schemas, deployment definitions, scripts, and repository documentation needed to build, verify, deploy, or maintain the software. Repositories should preserve enough dependency, build, and configuration information to recreate released software versions.

ngGONG will use a multi-repository strategy where repository boundaries reflect ownership, release cadence, deployment scope, and interface responsibility. Shared interface definitions, common schemas, deployment definitions, and reusable integration tooling should be maintained in repositories with clear ownership and compatibility controls. Each repository should have an accountable ngGONG software owner or lead.

Normal development should use a trunk-based workflow with short-lived branches. The primary branch should be protected from direct commits. Changes that affect delivered software, interfaces, tests, deployment behavior, or operational behavior should be merged through pull requests or an equivalent reviewed-change mechanism after required automated checks pass.

Controlled versions of software and interface definitions should be identifiable by tags, releases, commit secure hash algorithm (SHA) values, or equivalent version identifiers. Changes should reference related requirements, issues, defects, interface changes, releases, or deployments where applicable. Generated artifacts should be excluded from source control unless there is a clear reproducibility, delivery, review, or operational reason to include them.

# 8 Coding Standards and Review Practices

ngGONG will define coding standards by repository or technology area as implementation languages, frameworks, and runtime environments are selected. Where practical, repositories should use established external standards rather than project-specific style rules. Google style guides may be used as baseline references where they fit the selected language or ecosystem.

Coding standards should support readable, maintainable, testable software that can be operated and modified over the lifetime of ngGONG. Repository conventions should identify formatting, naming, static analysis, linting, dependency-management, documentation, and test expectations that apply to the software in that repository.

Changes that affect delivered software, interfaces, tests, deployment behavior, or operational behavior should be reviewed before merge. Review depth should match the risk and impact of the change. Small internal changes may need only a lightweight review, while changes to interfaces, safety-relevant behavior, deployment mechanisms, data products, or operations workflows require more careful review.

Code review should evaluate correctness, maintainability, test coverage, interface impact, operational impact, and consistency with repository conventions. Automated checks should be used where practical for formatting, linting, static analysis, dependency checks, and test execution. Human review remains responsible for design judgment, operational judgment, and risks that automated tools cannot reliably assess. AI-assisted review may be used when helpful, but it does not replace accountable human review.

Known exceptions to coding standards or review expectations should be documented where they affect delivered software, interfaces, operations, or long-term maintenance. New or modified tests should be included where practical for changed behavior, fixed defects, and interface changes.

# 9 Build, Integration, and Continuous Integration

Build, integration, and continuous integration practices provide the mechanism for building ngGONG software repeatably, integrating software early, and detecting defects before they accumulate. Each delivered software repository should provide documented build instructions and should build from version-controlled source and declared dependencies.

Controlled builds should record or preserve the source commit, dependency versions, build tools, build configuration, and build environment needed to reproduce the build. This information may be stored in the repository, continuous integration (CI) records, artifact metadata, or release records, depending on the repository and release model. Build outputs should be traceable to the source version, dependency versions, and build process that produced them. Generated interface artifacts should normally be treated as build products unless there is a clear reason to maintain them in source control.

Continuous integration should be used for pull requests and primary-branch changes where practical. CI checks should match the needs of the repository and may include compilation, unit tests, linting, static analysis, packaging, schema validation, and interface compatibility checks. Fast checks should be suitable for routine pull requests. More expensive integration tests may run on a schedule, on release candidates, or in dedicated integration contexts.

Interface definitions are build inputs and should be checked for compatibility with affected producers and consumers where practical. Integration should begin before all delivered hardware, site systems, or Data Management System services are available. Simulators, emulators, test harnesses, representative data, and mocked services may be used to support early integration.

Build, integration, and continuous integration practices should support integration across High Level Software, Controls, and the Data Management System, especially where command behavior, state transitions, Programmable Logic Controller (PLC) and hardware-control boundaries, telemetry, alarms, metadata, provenance, and data transfer are involved. CI results provide evidence that automated checks passed, but they do not by themselves approve software for site or operational deployment.

# 10 Verification and Validation

Software verification provides evidence that ngGONG software satisfies its allocated requirements. Software validation provides evidence that the software supports the intended operational and science use when integrated into the ngGONG system. This section describes the software evidence strategy; detailed test procedures, acceptance criteria, commissioning plans, and operations validation plans are maintained separately.

Each software requirement should have an identified verification method. Verification methods may include automated test, integration test, analysis, inspection, demonstration, or operational acceptance. The depth of verification should match the risk and impact of the requirement. Where practical, automated tests should be used for repeatable behavior, interface compatibility, regression testing, data validation, and failure handling.

Software verification will use multiple test levels where applicable, including unit, integration, system, interface, performance, fault and recovery, and acceptance testing. Early verification may use prototypes, simulations, emulators, representative data, mocked services, and test harnesses before all delivered hardware, station systems, or Data Management System services are available. Repository-specific test strategies may be used, but cross-subsystem behavior should be verified in representative integration contexts.

Verification is especially important where behavior crosses software subsystem or hardware-control boundaries. High Level Software and Controls verification should cover commands, state transitions, telemetry, alarms, timing behavior, interlocks, fault handling, and recovery behavior across the boundary between high-level software and low-level Programmable Logic Controller (PLC) and hardware control systems. Interfaces between High Level Software, Controls, and the Data Management System should be verified for data products, metadata, provenance, transfer status, completeness, integrity, and reduction context.

Verification evidence should be traceable to requirements, issues, builds, releases, and test results where applicable. Continuous integration results may provide part of this evidence, but passing automated checks alone does not necessarily establish readiness for acceptance, site deployment, or operations. Regression tests should be added for fixed defects where practical.

# 11 Release and Deployment Management

ngGONG release and deployment practices will identify what software was built, what combination of artifacts was qualified, what release is approved for deployment, and what release is installed in each environment or station. Release and deployment details may differ across High Level Software, Controls, and the Data Management System because those systems have different runtime models, hardware dependencies, operational constraints, and deployment mechanisms.

Release processes should separate build identity, artifact identity, release identity, and deployment state. A build identifies the output of one component build. An artifact identifies immutable content, such as a container image digest or published package. A release identifies a tested composition of artifacts and deployment configuration. Deployment state identifies the release approved or installed for a specific environment, channel, or station.

Released software should be traceable to source revisions, dependency versions, build records, configuration revisions, verification evidence, and deployment records. Promotion to an operational environment is a controlled deployment decision; it is not the same as passing continuous integration or publishing a build artifact.

For additional details about the various work products see their specific release and deployment management plans:

- Controls
  - SIS
- Data Management System
- Observatory Control System
- Station Control System.

## 11.1 Controls

The release and deployment strategy for Controls is not yet defined. Controls deployment may require different practices because it includes low-level Programmable Logic Controller (PLC), hardware-control, safety, interlock, vendor-tool, and station-maintenance constraints. Controls-specific release practices will be defined with the Controls design, hardware integration approach, and operations procedures.

## 11.2 DMS

The release and deployment strategy for the Data Management System (DMS) is not yet defined. DMS deployment may require different practices because it includes data pipelines, storage systems, processing environments, data-product compatibility, and operational data retention concerns. DMS-specific release practices will be defined with the DMS architecture, pipeline design, and operations model.

## 11.3 HLS - OCS

The release and deployment strategy for the High Level Software (HLS) Observatory Control System (OCS) is not yet defined. The OCS strategy will be developed as the OCS architecture, deployment model, operations role, and interface responsibilities mature.

## 11.4 HLS - SCS

The HLS Site Control System (SCS) release strategy uses immutable component artifacts and immutable system release manifests. Component builds produce artifacts such as container images or published packages. Human-readable tags or channels may be used for navigation, but deployment decisions should resolve to immutable artifact identifiers.

Each deployable SCS component should have a continuous integration pipeline that builds the component, runs the applicable automated checks, verifies relevant interfaces, publishes the artifact, and records the immutable artifact identity. A component-level candidate means that the component passed its own checks and is eligible for inclusion in an SCS release candidate. It does not mean that the component has passed SCS system qualification or is approved for production deployment.

An SCS deployment repository should define deployment configuration and compose SCS release candidates. The release-composition process should resolve each required component to an immutable artifact identity, identify the deployment-configuration revision, generate an immutable release manifest, assign a unique SCS release identifier, and archive the manifest before deployment or qualification. The process should fail rather than create an incomplete release when required components, configuration, or provenance cannot be resolved.

The SCS release manifest is the system-level release artifact. It records the exact component artifacts and deployment configuration that were qualified. At minimum, the manifest should identify the SCS release, the creating pipeline or process, the deployment repository revision, component artifact identities, component source revisions, configuration revisions, applicable schema or migration levels, and available provenance records. The manifest should be machine-validated.

SCS qualification should test the exact release manifest under evaluation. New component candidates published while qualification is in progress should not change the release being tested. If qualification succeeds, an authorized promotion process may advance the applicable deployment channel to the qualified release. Promotion should use the same manifest and artifact identities that passed qualification; it should not rebuild component artifacts or create a replacement manifest.

SCS deployment channels are mutable pointers to immutable release manifests. A channel may represent a candidate release, a production release, or a station-specific approved release. Stations should deploy only the release approved for their applicable channel and should not deploy from component-level candidate tags, production tags, or latest tags. Before deploying, a station should confirm that the selected release remains approved and has not been revoked.

Promotion, revocation, and rollback are separate operations. Promotion advances a channel to a qualified release. Revocation prevents future deployment of a defective or withdrawn release. Rollback changes desired or installed state to a prior known-good release when that is operationally safe. Revoked release manifests, artifacts, evidence, and deployment history should be retained rather than modified or deleted.

Rollback should not be assumed to be safe for stateful components. Database migrations, persisted data formats, protocol state, and controller configuration may require a forward fix or a specific recovery procedure. Stateful SCS components should declare whether rollback is supported, requires state restoration, or is not supported for a given release transition.

SCS release, channel, qualification, revocation, rollback, and station deployment records should use stable identifiers. The records should make it possible to determine which source revisions produced each artifact, which artifacts formed each release, which configuration was applied, which tests qualified the release, who or what promoted or revoked it, and which release each station installed.

# 12 Defect and Issue Management

ngGONG software issues will be tracked in a common issue-tracking system where practical. The project has not yet selected the final software issue-tracking tool. Candidate tools include GitHub Issues and Atlassian Jira. The selected approach should support traceability across software work, requirements, pull requests, builds, releases, verification evidence, and deployment records.

Software issue tracking should include defects, enhancement requests, technical tasks, interface changes, test failures, integration problems, and operational software problems. Issues should contain enough information to reproduce, evaluate, prioritize, assign, and close the work. Each issue should identify the affected subsystem, repository, interface, deployment context, or test environment where applicable.

Defects should be classified by severity and impact. Relevant impact areas include operations, data products, safety or hardware-control behavior, interfaces, integration, deployment, and maintenance. Each issue should have an accountable software owner or lead for triage and resolution. Cross-subsystem issues should be triaged with the affected leads, especially where behavior crosses the High Level Software and Controls boundary or the High Level Software, Controls, and Data Management System boundary.

Fixed defects should include regression tests or other verification evidence where practical. Known limitations, accepted defects, deferred fixes, and waivers should be recorded when they affect delivered software, operations, interfaces, or long-term maintenance. Defect history should be preserved well enough to support release decisions, operational troubleshooting, and maintenance planning.

Software defects should be escalated into project-level risk management, configuration management, or change-control processes when their impact warrants project-level handling. Recurring or systemic defects should be treated as evidence that requirements, design, tests, tooling, or operational procedures may need improvement.

# 13 Cybersecurity Coordination

- Cybersecurity requirements, controls, assessment expectations, and incident-response procedures are defined through project-level cybersecurity or systems engineering processes, not by this Software Development Plan.
- Software teams coordinate with the project cybersecurity owner to apply applicable requirements to repositories, dependencies, continuous integration and deployment mechanisms, credentials, network interfaces, logging, and operational software.
- Software designs and releases identify security-relevant assumptions or dependencies where they affect architecture, interfaces, deployment, or operations.
- Cybersecurity findings that require software changes are tracked through the selected software issue-tracking process, with escalation to project-level risk management or change control when warranted.
- Detailed security control selection, threat analysis, authorization, vulnerability management policy, and incident-response procedures are owned by the responsible cybersecurity lead or team.

# 14 Transition to Operations and Software Maintenance

- Software transition-to-operations planning will mature with the ngGONG operations concept, commissioning plan, site deployment model, and staffing model.
- Software delivered for operations should include the documentation, build records, release records, configuration information, verification evidence, and known-issue records needed to support operational use and maintenance.
- Operational software should provide appropriate observability, including logs, status, health information, metrics, alarms, or diagnostics needed for troubleshooting and maintenance.
- Software maintenance planning should address defect correction, dependency updates, interface changes, security updates, configuration changes, and compatibility with deployed hardware, site systems, and data-processing environments.
- Transition planning should identify which software components are maintained by ngGONG staff, contractors, vendors, or operations personnel.
- Operations-facing software should support controlled update, rollback, recovery, and configuration-management practices appropriate to its role and risk.
- Lessons learned from integration, commissioning, and early operations should feed back into requirements, design, tests, documentation, and maintenance priorities.